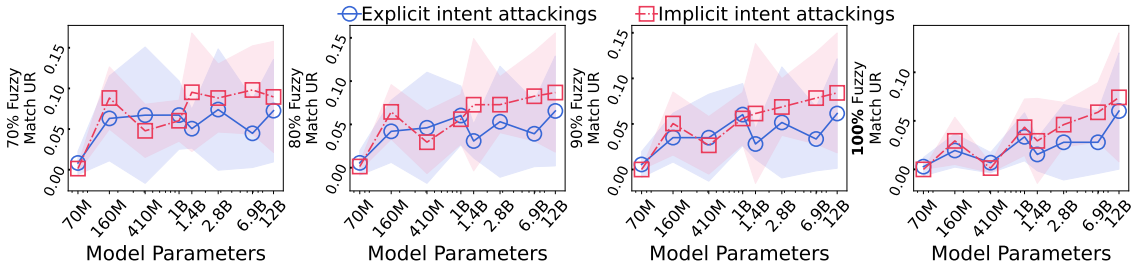

Figure 1: Prompt extraction performance across different model sizes.

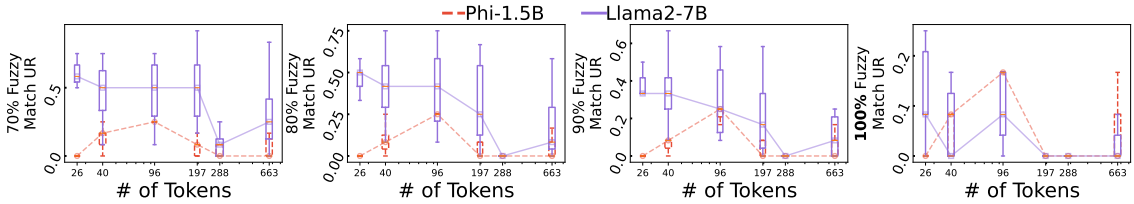

Figure 2: The relationship between prompt length and prompt uncover rate.

between implicit and explicit intent attacks is not obvious compared to the following experiments (e.g., Table 3). This might be because uncovered rates of prompt extraction usually depend on the effectiveness of instruction following, and Pythia models are not specifically designed for improving instruction-following capacities.

3.2 Prompt Length

We also evaluate LLMs across different sequence lengths. To this end, we split the whole range of sequence lengths into six exponentially increasing intervals, spanning from 2^4 to 2^{10}. Within each interval, we sample six prompts. We compute the minimum, maximum, and average uncovered rate for each interval, as shown in Figure 2 and 10.

When comparing Llama2-7B with Phi-1.5B, it is clear that the UR drops gradually with the increase of sequence length under normalized metrics such as ρ-fuzzy similarity. For 100%-fuzzy match, it is observed that longer prompts pose greater challenges for extraction. In contrast, the UR of short text fragments, as shown in Figure 10, rises steadily with the increase in sequence length for 7 billion Llama-2 models. Specifically, for prompts with an average token number of 663, the UR of 12-gram even approaches nearly 100%. Another notable finding from Figure 2 is the high variances of all LLMs. In particular, the larger the LLMs, the higher the variance. This means LLMs are sensitive to prompts and user inputs. Since our attack prompts are derived from rephrasing (see Section 2.2), this phenomenon is more eminent in real attacks.

3.3 Instructions vs. Function Callings.

Finally, we study whether different forms of prompts yield different URs under the same attacks. We divide the whole dataset into two categories: *unstructured natural language texts* (i.e., instructions), and the *JSON-format function callings*. For a fair comparison, we sample prompts from both categories with similar lengths, varying from 256 to 1,024. Illustrated by Figure 3, it is observed that there is no statistical distinction in short fragments extraction between these two types of prompts. However, from ρ-fuzzy experiments it is clear that the UR of natural language instructions decreases faster than that of function callings. While the 100%-fuzzy extraction rate reaches almost zero for most of the natural language prompts, function callings remain possible for extraction and exhibit a decreasing trend of UR with increasing tokens. This phenomenon, which we call "prompt memorization", is somewhat counterintuitive, because prompts that make sense and are easy to understand should lead to a high UR, as opposed to those long and bizarre prompts. The underlying explanation of this phenomenon will be elaborated in Section 4.

4 Empirical Analysis

The prompt memorization phenomenon, as we discussed in Section 3.3, describes the vulnerability of LLMs to translate their prompts accurately and precisely to users. To investigate potential properties of prompts and underlying mechanisms in LLMs that lead to such a memorization phenomenon, we

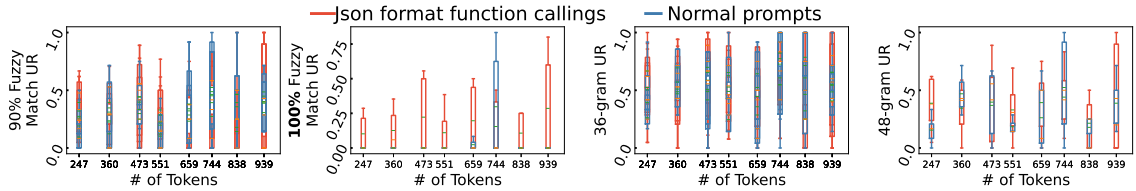

Figure 3: Comparison of prompt extraction between two types of prompts: function callings and natural language.

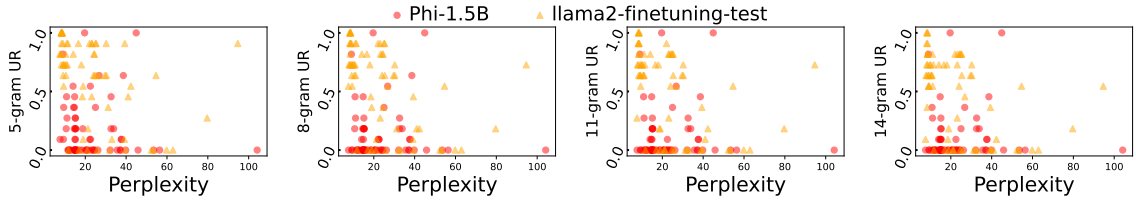

Figure 4: The distribution of uncover rate across varying perplexities.

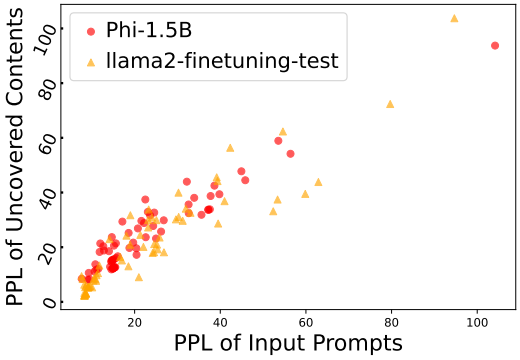

Figure 5: Correlation between the perplexities of prompts and the perplexities of extracted texts.

provide two explanations, including *convincing premise* and the *parallel translation*, as shown in Section 4.1 and 4.2, respectively.

4.1 Which Prompts are More Susceptible to Leakage under Adversarial Prompts?

We first propose the *convincing premise*, positing that the *familiarity* of a language model with user prompts significantly influences the efficacy of prompt extraction. Specifically, the convincing premise suggests that those prompts with which language models are familiar are memorized and extracted easily, and the likelihood value (measured by *perplexity* in Equation 12) of prompts serves as a *premise* for prompt memorization. To verify this premise, we analyze the correlation among the *uncovered rates*, *prompts' perplexity*, and the *generated prompts' perplexity*.

We first study the relationship between the prompt's perplexity and uncovered rates. Illustrated by Figure 4, we plot the distribution of 748 cases under 11 implicit intent attack inputs (i.e., every point in the figure is the average value of these cases) on both Phi-1.5B and LLama2-7B. It is clear that prompts with a lower perplexity tend to yield a much higher uncovered rate, though most of the prompts in Phi-1.5B are located at the bottom left quadrant (i.e., low perplexity with poor UR). We also observe that the perplexity of most of the prompts is below 60, indicating that most of the prompts crafted by humans also make sense to LLMs, and those unfamiliar prompts with a perplexity greater than 50 truly result in a relatively lower UR. Besides, the UR of all prompts gets smaller for a longer text fragments extraction, which is consistent with the results in Section 3.2.

We then explore the perplexity correlation between the prompts and the recovered prompts, so as to explore the underlying mechanism of prompt extraction. To our surprise, in contrast to the results in Figure 4 where points on Phi-1.5B are usually located at the bottom-left quadrant, the results from Phi-1.5B and Llama2-7B are quite consistent. Both models exhibit a strong positive correlation between the prompts and recovered prompts, with Spearman correlation scores of 0.89 and 0.92 for Phi and LLama2, respectively. Meanwhile, the perplexity of recovered prompts measured by these two models shows no significant difference between each other. This phenomenon demonstrates that while the *familiarity* of the input prompts serves as a *premise*, the confidence of *generated* prompts, however, cannot be used as a metric or corresponding criterion for prompt extraction.

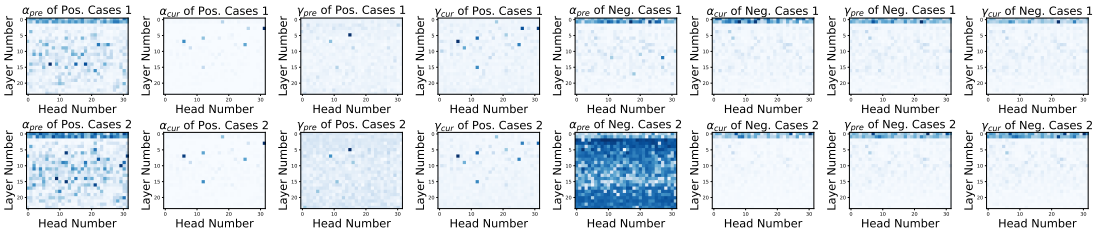

Figure 6: Experiment results of four proposed indicators (α_{pre}, α_{cur}, γ_{pre}, and γ_{cur}) in prompt extraction among three successful (Pos.) and three failed (Neg.) cases.

4.2 How Does an LLM Leak Prompts?

Our second explanation for accurate prompt memorization comes from the *attention mechanism* of LLMs, where there are some strong weight connections in attention matrices, so that the influence of prompts in the translation procedure is unusually reinforced. To verify this, a straightforward approach is to visualize the attention matrices at each layer and head, and evaluate these matrices through human crowdsourcing. However, unlike previous pre-trained models like BERT (Devlin et al., 2019), visualizing and reviewing attention matrices for large language models becomes challenging due to the large number (e.g., 2^{10} for 6.9 billion models) of matrices to check, i.e., $d_l \times d_h$ matrices for a d_l-layer d_h-head model.

Therefore, to measure the prompt memorization of LLMs, we turn to designing some automatic and reference-free indicators as below:

• *Single Prompt Linking Indicator (SPLIt)*: the average[1] attention value from token x^p_{t-1} in prompt p to its corresponding next token $x^{p_g}_t$ in the generated prompt p_g, i.e.,

$$\alpha^{(l,h)}_{pre} = \prod_{t=1}^{N_p} Attn^{(l,h)}(x^p_{t-1}, x^{p_g}_t)^{\frac{1}{N_p}}, \quad (5)$$

where N_p is the length of the prompt p, and $Attn^{(l,h)}(x_a, x_b)$ denotes the h-th head attention weights in the l-th layer from the token x_a to x_b. Based on $\alpha^{(l,h)}_{pre}$, we define the *directly connected attention weight*, i.e., the attention from token x^p_t to its corresponding token $x^{p_g}_t$, as

$$\alpha^{(l,h)}_{cur} = \prod_{t=1}^{N_p} Attn^{(l,h)}(x^p_t, x^{p_g}_t)^{\frac{1}{N_p}}. \quad (6)$$

• *Normalized SPLIt (N-SPLIt)*: the normalized version of SPLIt, i.e.,

$$\gamma^{(l,h)}_{pre} = \prod_{t=1}^{N_p} \sqrt[N_p]{\frac{Attn^{(l,h)}(x^p_{t-1}, x^{p_g}_t)}{\sum_{j,k \in N_p} Attn^{(l,h)}(x^p_j, x^{p_g}_k)}}, \quad (7)$$

and the normalized $\alpha^{(n,l)}_{cur}$ can be formatted as

$$\gamma^{(l,h)}_{cur} = \prod_{t=1}^{N_p} \sqrt[N_p]{\frac{Attn^{(l,h)}(x^p_t, x^{p_g}_t)}{\sum_{j,k \in N_p} Attn^{(l,h)}(x^p_j, x^{p_g}_k)}}. \quad (8)$$

Based on indicators α_{pre}, α_{cur}, γ_{pre}, and γ_{cur}, we sample and evaluate both the successful and unsuccessful prompt extraction cases in previous experiments. Specifically, we consider prompts extracted exactly as successful samples, and prompts with 0-fuzzy matched rate as failure cases. The heatmaps depicting these two indicators into both successful and failure cases are shown in Figure 6.

From Figure 6, the α_{pre} values for six samples exhibit no statistically significant correlation with their categories. However, its normalized version, the γ_{pre}, shows an obvious difference between successful and failed samples. For the latter, the γ_{pre} of all attention-heads except the first two layers is close to zero, while in the former cases, there exist some attention heads (e.g., layer-5, head-15 in cases 1 and 2, and layer-6,head-12 in case 3) achieving a relatively higher score. This means that there are some direct but unstable next-token connections from prompts to generated prompts. Moreover, in memorized cases we see a significant difference from α_{cur} and γ_{cur}: there are $8 \sim 10$ *abnormal* attention heads activated and exhibiting unusually higher α_{cur} and γ_{cur} scores than others. This phenomenon demonstrates that LLMs can not only "remind" a token by its context, but also "translate" (i.e., copy) a token directly, which could be the key explanation for prompt memorization. Besides, by observing the activated attention heads in these three successful cases, we find that the direct and strong connections in prompt translation are particularly stable, i.e., they are limited on some fixed and specific attention heads even under different prompts. This phenomenon implies that *the ability of prompt memorization may be learned at the pre-training stage, and then these translation-functioned heads can be "activated" under certain user inputs.* In addition, these

[1]The theoretical analysis of why we use *geometric mean* rather than the *arithmetic mean* can be found in Appendix C.